Problem Statement

Data Set: Fire Department Calls for Service

Location: /databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv

What are the requirements

1. Load the given data file and create a Spark data frame.
2. Use the Spark data frame to answer the following questions.
    1. How many distinct types of calls were made to the fire department?
    2. What are distinct types of calls made to the fire department?
    3. Find out all responses or delayed times greater than 5 mins?
    4. What were the most common call types?
    5. What zip codes accounted for the most common calls?
    6. What San Francisco neighborhoods are in the zip codes 94102 and 94103
    7. What was the sum of all calls, average, min, and max of the call response times?
    8. How many distinct years of data are in the CSV file?
    9. What week of the year in 2018 had the most fire calls?
    10. What neighborhoods in San Francisco had the worst response time in 2018?

02-spark-dataframe-demo - Dat... | Spark SQL — PySpark 3.2.1 docu...

spark.apache.org/docs/latest/api/python/reference/pyspark.sql.html

Spark

Getting Started | User Guide | **API Reference** | Development | Migration Guide

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- pyspark.sql.DataFrame
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

Spark SQL

Core Classes

`SparkSession`(sparkContext[, jsparkSession])	
`Catalog`(sparkSession)	
`DataFrame`(jdf, sql_ctx)	
`Column`(jc)	A column in a DataFrame.
`Row`	A row in `DataFrame`.
`GroupedData`(jgd, df)	A set of methods for aggregations on a `DataFrame`, created by `DataFrame.groupBy()`.

1. pyspark.sql.SparkSession
2. pyspark.sql.DataFrame ←
3. pyspark.sql.DataFrameReader
4. pyspark.sql.DataFrameWriter
5. pyspark.sql.Row
6. pyspark.sql.Column
7. pyspark.sql.types
8. pyspark.sql.GroupedData
9. pyspark.sql.Window
10. pyspark.sql.Catalog
11. pyspark.sql.conf.
12. pyspark.sql.functions
13. pyspark.sql.DataFrameNaFunctions
14. pyspark.sql.DataFrameStatFunctions

On this page

- e Classes
- rk Session APIs
- figuration
- ut and Output
- aFrame APIs
- umn APIs
- a Types
- w
- ctions
- dow
- uping
- alog APIs

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- pyspark.sql.DataFrame
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

pyspark.sql.SparkSession

class pyspark.sql.**SparkSession**(*sparkContext, jsparkSession=None*) [source]

The entry point to programming Spark with the Dataset and DataFrame API.

A SparkSession can be used create `DataFrame`, register `DataFrame` as tables, execute SQL over tables, cache tables, and read parquet files. To create a `SparkSession`, use the following builder pattern:

builder

A class attribute having a `Builder` to construct `SparkSession` instances.

Examples

```
>>> spark = SparkSession.builder \
...     .master("local") \
...     .appName("Word Count") \
...     .config("spark.some.config.option", "some-value") \
...     .getOrCreate()
```

```
>>> from datetime import datetime
>>> from pyspark.sql import Row
>>> spark = SparkSession(sc)
>>> allTypes = sc.parallelize([Row(i=1, s="string", d=1.0, l=1,
```

Search the docs ...

Spark SQL
- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- pyspark.sql.DataFrame
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

Methods

`createDataFrame`(data[, schema, ...])	Creates a `DataFrame` from an `RDD`, a list or a `pandas.DataFrame`.
`getActiveSession`()	Returns the active `SparkSession` for the current thread, returned by the builder
`newSession`()	Returns a new `SparkSession` as new session, that has separate SQLConf, registered temporary views and UDFs, but shared `SparkContext` and table cache.
`range`(start[, end, step, numPartitions])	Create a `DataFrame` with single `pyspark.sql.types.LongType` column named `id`, containing elements in a range from `start` to `end` (exclusive) with step value `step`.
`sql`(sqlQuery)	Returns a `DataFrame` representing the result of the given query.
`stop`()	Stop the underlying `SparkContext`.
`table`(tableName)	Returns the specified table as a `DataFrame`.

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- pyspark.sql.DataFrame
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

Attributes

`builder`	A class attribute having a `Builder` to construct `SparkSession` instances.
`catalog`	Interface through which the user may create, drop, alter or query underlying databases, tables, functions, etc.
`conf`	Runtime configuration interface for Spark.
`read`	Returns a `DataFrameReader` that can be used to read data in as a `DataFrame`.
`readStream`	Returns a `DataStreamReader` that can be used to read data streams as a streaming `DataFrame`.
`sparkContext`	Returns the underlying `SparkContext`.
`streams`	Returns a `StreamingQueryManager` that allows managing all the `StreamingQuery` instances active on *this* context.
`udf`	Returns a `UDFRegistration` for UDF registration.
`version`	The version of Spark on which this application is running.

02-spark-dataframe-demo - Dat... | pyspark.sql.SparkSession — PySp...

spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html

Spark

Getting Started | User Guide | **API Reference** | Development | Migration Guide

Search the docs ...

park.sql.SparkSession.version
park.sql.conf.RuntimeConfig
park.sql.DataFrameReader.csv
park.sql.DataFrameReader.format
park.sql.DataFrameReader.jdbc
park.sql.DataFrameReader.json
park.sql.DataFrameReader.load
park.sql.DataFrameReader.option
park.sql.DataFrameReader.options
park.sql.DataFrameReader.orc
park.sql.DataFrameReader.parquet
park.sql.DataFrameReader.schema
park.sql.DataFrameReader.table
park.sql.DataFrameWriter.bucketBy
park.sql.DataFrameWriter.csv
park.sql.DataFrameWriter.format

Attributes

`builder`	A class attribute having a `Builder` to construct `SparkSession` instances.
`catalog`	Interface through which the user may create, drop, alter or query underlying databases, tables, functions, etc.
`conf`	Runtime configuration interface for Spark.
`read`	Returns a `DataFrameReader` that can be used to read data in as a `DataFrame`.
`readStream`	Returns a `DataStreamReader` that can be used to read data streams as a streaming `DataFrame`.
`sparkContext`	Returns the underlying `SparkContext`.
`streams`	Returns a `StreamingQueryManager` that allows managing all the `StreamingQuery` instances active on *this* context.
`udf`	Returns a `UDFRegistration` for UDF registration.
`version`	The version of Spark on which this application is running.

02-spark-dataframe-demo - Dat × pyspark.sql.SparkSession — PySp ×

spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html

Spark

Getting Started User Guide **API Reference** Development Migration Guide

Search the docs ...

k.sql.SparkSession.version
k.sql.conf.RuntimeConfig
k.sql.DataFrameReader.csv
k.sql.DataFrameReader.format
k.sql.DataFrameReader.jdbc
k.sql.DataFrameReader.json
k.sql.DataFrameReader.load
k.sql.DataFrameReader.option
k.sql.DataFrameReader.options
k.sql.DataFrameReader.orc
k.sql.DataFrameReader.parquet
k.sql.DataFrameReader.schema
k.sql.DataFrameReader.table
k.sql.DataFrameWriter.bucketBy
k.sql.DataFrameWriter.csv
k.sql.DataFrameWriter.format

Attributes

`builder`	A class attribute having a `Builder` to construct `SparkSession` instances.
`catalog`	Interface through which the user may create, drop, alter or databases, tables, functions, etc.
`conf`	Runtime configuration interface for Spark.
`read`	Returns a `DataFrameReader` that can be used to read data in
`readStream`	Returns a `DataStreamReader` that can be used to read data `DataFrame`.
`sparkContext`	Returns the underlying `SparkContext`.
`streams`	Returns a `StreamingQueryManager` that allows managing all instances active on *this* context.
`udf`	Returns a `UDFRegistration` for UDF registration.
`version`	The version of Spark on which this application is running.

1. DataFrameReader.csv
2. DataFrameReader.json
3. DataFrameReader.parquet
4. DataFrameReader.orc
5. DataFrameReader.jdbc
6. DataFrameReader.table
7. DataFrameReader.format
8. DataFrameReader.option
9. DataFrameReader.options
10. DataFrameReader.schema
11. DataFrameReader.load

02-spark-dataframe-demo - Dat × pyspark.sql.DataFrameReader.cs ×

community.cloud.databricks.com/?o=946780288864704#notebook/2787702214819532/command/116686210829263

02-spark-dataframe-demo Python

demo-cluster

Cmd 1

```python
fire_df = spark.read \
            .format("csv") \
            .option("header", "true") \
            .option("inferSchema", "true") \
            .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

Cmd 2

Python

```python
fire_df = spark.read \
            .csv("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv",
                header="true",
                inferSchema="true")
```

Shift+Enter to run

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- **pyspark.sql.DataFrame**
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

pyspark.sql.DataFrame

class pyspark.sql.**DataFrame**(*jdf, sql_ctx*) **[source]**

A distributed collection of data grouped into named columns.

A `DataFrame` is equivalent to a relational table in Spark SQL, and can be created using various functions in `SparkSession`:

```
people = spark.read.parquet("...")
```

Once created, it can be manipulated using the various domain-specific-language (DSL) functions defined in: `DataFrame`, `Column`.

To select a column from the `DataFrame`, use the apply method:

```
ageCol = people.age
```

A more concrete example:

```
# To create DataFrame using SparkSession
people = spark.read.parquet("...")
department = spark.read.parquet("...")
```

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- **pyspark.sql.DataFrame**
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

Methods

`agg`(*exprs)	Aggregate on the entire `DataFrame` without groups (shorthand for `df.groupBy().agg()`).
`alias`(alias)	Returns a new `DataFrame` with an alias set.
`approxQuantile`(col, probabilities, relativeError)	Calculates the approximate quantiles of numerical columns of a `DataFrame`.
`cache`()	Persists the `DataFrame` with the default storage level (*MEMORY_AND_DISK*).
`checkpoint`([eager])	Returns a checkpointed version of this `DataFrame`.
`coalesce`(numPartitions)	Returns a new `DataFrame` that has exactly *numPartitions* partitions.
`colRegex`(colName)	Selects column based on the column name specified as a regex and returns it as `Column`.
`collect`()	Returns all the records as a list of `Row`.

Search the docs ...

Spark SQL
pyspark.sql.SparkSession
pyspark.sql.Catalog
pyspark.sql.DataFrame
pyspark.sql.Column
pyspark.sql.Row
pyspark.sql.GroupedData
pyspark.sql.PandasCogroupedOps
pyspark.sql.DataFrameNaFunction
pyspark.sql.DataFrameStatFunctio
pyspark.sql.Window
pyspark.sql.SparkSession.builder.a
pyspark.sql.SparkSession.builder.c
pyspark.sql.SparkSession.builder.e
pyspark.sql.SparkSession.builder.g

`createOrReplaceGlobalTempView`(name)	Creates or replaces a global temporary view using the given name.
`createOrReplaceTempView`(name)	Creates or replaces a local temporary view with this `DataFrame`.
`createTempView`(name)	Creates a local temporary view with this `DataFrame`.
`crossJoin`(other)	Returns the cartesian product with another `DataFrame`.
`crosstab`(col1, col2)	Computes a pair-wise frequency table of the given columns.
`cube`(*cols)	Create a multi-dimensional cube for the current `DataFrame` using the specified columns, so we can run aggregations on them.
`describe`(*cols)	Computes basic statistics for numeric and string columns.
`distinct`()	Returns a new `DataFrame` containing the distinct rows in this `DataFrame`.

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- **pyspark.sql.DataFrame**
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

	removed, optionally only considering certain columns.
`drop_duplicates`([subset])	`drop_duplicates()` is an alias for `dropDuplicates()`.
`dropna`([how, thresh, subset])	Returns a new `DataFrame` omitting rows with null values.
`exceptAll`(other)	Return a new `DataFrame` containing rows in this `DataFrame` but not in another `DataFrame` while preserving duplicates.
`explain`([extended, mode])	Prints the (logical and physical) plans to the console for debugging purpose.
`fillna`(value[, subset])	Replace null values, alias for `na.fill()`.
`filter`(condition)	Filters rows using the given condition.
`first`()	Returns the first row as a `Row`.
`foreach`(f)	Applies the `f` function to all `Row` of this `DataFrame`.

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- **pyspark.sql.DataFrame**
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

	`DataFrame`.
`freqItems`(cols[, support])	Finding frequent items for columns, possibly with false positives.
`groupBy`(*cols)	Groups the `DataFrame` using the specified columns, so we can run aggregation on them.
`groupby`(*cols)	`groupby()` is an alias for `groupBy()`.
`head`([n])	Returns the first `n` rows.
`hint`(name, *parameters)	Specifies some hint on the current `DataFrame`.
`inputFiles`()	Returns a best-effort snapshot of the files that compose this `DataFrame`.
`intersect`(other)	Return a new `DataFrame` containing rows only in both this `DataFrame` and another `DataFrame`.
`intersectAll`(other)	Return a new `DataFrame` containing rows in both this `DataFrame` and another `DataFrame` while preserving duplicates.

Search the docs ...

Spark SQL

- pyspark.sql.SparkSession
- pyspark.sql.Catalog
- **pyspark.sql.DataFrame**
- pyspark.sql.Column
- pyspark.sql.Row
- pyspark.sql.GroupedData
- pyspark.sql.PandasCogroupedOps
- pyspark.sql.DataFrameNaFunction
- pyspark.sql.DataFrameStatFunctio
- pyspark.sql.Window
- pyspark.sql.SparkSession.builder.a
- pyspark.sql.SparkSession.builder.c
- pyspark.sql.SparkSession.builder.e
- pyspark.sql.SparkSession.builder.g

`intersect`(other)	Return a new `DataFrame` containing rows only in both this `DataFrame` and another `DataFrame`.
`intersectAll`(other)	Return a new `DataFrame` containing rows in both this `DataFrame` and another `DataFrame` while preserving duplicates.
`isLocal`()	Returns `True` if the `collect()` and `take()` methods can be run locally (without any Spark executors).
`join`(other[, on, how])	Joins with another `DataFrame`, using the given join expression.
`limit`(num)	Limits the result count to the number specified.
`localCheckpoint`([eager])	Returns a locally checkpointed version of this `DataFrame`.
`mapInPandas`(func, schema)	Maps an iterator of batches in the current `DataFrame` using a Python native function that takes and outputs a pandas DataFrame, and returns the result as a `DataFrame`.
`orderBy`(*cols, **kwargs)	Returns a new

Search the docs ...

Spark SQL

pyspark.sql.SparkSession
pyspark.sql.Catalog
pyspark.sql.DataFrame
pyspark.sql.Column
pyspark.sql.Row
pyspark.sql.GroupedData
pyspark.sql.PandasCogroupedOps
pyspark.sql.DataFrameNaFunction
pyspark.sql.DataFrameStatFunctio
pyspark.sql.Window
pyspark.sql.SparkSession.builder.a
pyspark.sql.SparkSession.builder.c
pyspark.sql.SparkSession.builder.e
pyspark.sql.SparkSession.builder.g

	column(s).
`persist`([storageLevel])	Sets the storage level to persist the contents of the `DataFrame` across operations after the first time it is computed.
`printSchema`()	Prints out the schema in the tree format.
`randomSplit`(weights[, seed])	Randomly splits this `DataFrame` with the provided weights.
`registerTempTable`(name)	Registers this `DataFrame` as a temporary table using the given name.
`repartition`(numPartitions, *cols)	Returns a new `DataFrame` partitioned by the given partitioning expressions.
`repartitionByRange`(numPartitions, *cols)	Returns a new `DataFrame` partitioned by the given partitioning expressions.
`replace`(to_replace[, value, subset])	Returns a new `DataFrame` replacing a value with another value.
`rollup`(*cols)	Create a multi-dimensional rollup for the current

Search the docs ...

Spark SQL

pyspark.sql.SparkSession
pyspark.sql.Catalog
pyspark.sql.DataFrame
pyspark.sql.Column
pyspark.sql.Row
pyspark.sql.GroupedData
pyspark.sql.PandasCogroupedOps
pyspark.sql.DataFrameNaFunction
pyspark.sql.DataFrameStatFunctio
pyspark.sql.Window
pyspark.sql.SparkSession.builder.a
pyspark.sql.SparkSession.builder.c
pyspark.sql.SparkSession.builder.e
pyspark.sql.SparkSession.builder.g

`sampleBy`(col, fractions[, seed])	Returns a stratified sample without replacement based on the fraction given on each stratum.
`select`(*cols)	Projects a set of expressions and returns a new `DataFrame`.
`selectExpr`(*expr)	Projects a set of SQL expressions and returns a new `DataFrame`.
`semanticHash`()	Returns a hash code of the logical query plan against this `DataFrame`.
`show`([n, truncate, vertical])	Prints the first `n` rows to the console.
`sort`(*cols, **kwargs)	Returns a new `DataFrame` sorted by the specified column(s).
`sortWithinPartitions`(*cols, **kwargs)	Returns a new `DataFrame` with each partition sorted by the specified column(s).
`subtract`(other)	Return a new `DataFrame` containing rows in this `DataFrame` but not in another `DataFrame`.
`summary`(*statistics)	Computes specified statistics for numeric and

Search the docs ...

Spark SQL

pyspark.sql.SparkSession
pyspark.sql.Catalog
pyspark.sql.DataFrame
pyspark.sql.Column
pyspark.sql.Row
pyspark.sql.GroupedData
pyspark.sql.PandasCogroupedOps
pyspark.sql.DataFrameNaFunction
pyspark.sql.DataFrameStatFunctio
pyspark.sql.Window
pyspark.sql.SparkSession.builder.a
pyspark.sql.SparkSession.builder.c
pyspark.sql.SparkSession.builder.e
pyspark.sql.SparkSession.builder.g

`toJSON`([use_unicode])	Converts a `DataFrame` into a `RDD` of string.
`toLocalIterator`([prefetchPartitions])	Returns an iterator that contains all of the rows in this `DataFrame`.
`toPandas`()	Returns the contents of this `DataFrame` as Pandas `pandas.DataFrame`.
`to_koalas`([index_col])	
`to_pandas_on_spark`([index_col])	Converts the existing DataFrame into a pandas-on-Spark DataFrame.
`transform`(func)	Returns a new `DataFrame`.
`union`(other)	Return a new `DataFrame` containing union of rows in this and another `DataFrame`.
`unionAll`(other)	Return a new `DataFrame` containing union of rows in this and another `DataFrame`.
`unionByName`(other[, allowMissingColumns])	Returns a new `DataFrame` containing union of rows in this and another `DataFrame`.

Search the docs ...

Spark SQL

pyspark.sql.SparkSession
pyspark.sql.Catalog
pyspark.sql.DataFrame
pyspark.sql.Column
pyspark.sql.Row
pyspark.sql.GroupedData
pyspark.sql.PandasCogroupedOps
pyspark.sql.DataFrameNaFunction
pyspark.sql.DataFrameStatFunctio
pyspark.sql.Window
pyspark.sql.SparkSession.builder.a
pyspark.sql.SparkSession.builder.c
pyspark.sql.SparkSession.builder.e
pyspark.sql.SparkSession.builder.g

`withColumnRenamed`(existing, new)	Returns a new `DataFrame` by renaming an existing column.
`withWatermark`(eventTime, delayThreshold)	Defines an event time watermark for this `DataFrame`.
`writeTo`(table)	Create a write configuration builder for v2 sources.

Attributes

`columns`	Returns all column names as a list.
`dtypes`	Returns all column names and their data types as a list.
`isStreaming`	Returns `True` if this `DataFrame` contains one or more sources that continuously return data as it arrives.
`na`	Returns a `DataFrameNaFunctions` for handling missing values.
`rdd`	Returns the content as an `pyspark.RDD` of `Row`.
`schema`	Returns the schema of this `DataFrame` as a `pyspark.sql.types.StructType`.